

关系代数 + SQL 综合 —— 练习题解析

板块一：关系代数·基础

第 1 题

答案：

- $(1) \pi \text{ 姓名, 学号 } (\sigma \text{ 所在系='计算机系' } \wedge \text{ 性别='女' } \wedge \text{ 入学年份} \geq 2020 (S))$

解析：这题考的是单表上 π/σ 的“过滤+投影”分工—— σ 负责“留谁”， π 负责“取哪几列”。

(1) 查询计算机系 2020 年及以后入学的女生姓名与学号

关系代数表达式标准写法是从内到外先 σ 后 π (先过滤行、再取列)，也可以嵌套写：

$\pi \text{ 姓名, 学号 } (\sigma \text{ 所在系='计算机系' } \wedge \text{ 性别='女' } \wedge \text{ 入学年份} \geq 2020 (S))$

子表达式分工：

- $\sigma \text{ 所在系='计算机系' } \wedge \text{ 性别='女' } \wedge \text{ 入学年份} \geq 2020 (S)$ ：从学生表 S 中过滤出满足 3 个条件 (计算机系、女生、2020 后入学) 的行
- $\pi \text{ 姓名, 学号 } (...)$ ：从过滤结果中只取姓名和学号两列
- 三个条件用合取符号 $\wedge$ 连接 (不能漏任何一个，否则会引入不满足条件的学生)

易错点：

- 把“ ≥ 2020 ”写成“ > 2020 ”→ 错。题目说“2020 年及以后”，包含 2020 本身，必须用 $\geq$ 。
- 把 $\wedge$ 写成 $\vee$ (析取) → 错。三个条件是“同时满足”，不是“任一即可”。
- 投影列写成 $\pi * (...)$ → 错。题目只要求姓名与学号，必须显式列出。
- 把 σ 写在 π 外面 → 错。 π 在 σ 外面时先投影再选择，但单属性选择时结果相同；多属性时若先投影会丢列导致选择困难。标准写法是 σ 在 π 之内。

金句： π 和 σ 是孪生兄弟—— σ 负责“留谁” (行筛选)， π 负责“取哪几列” (列投影)；先 σ 后 π 是单表查询的标准顺序。

第 2 题

答案：

- $(1) \pi \text{ 商品名, 单价 } (\sigma \text{ 城市} \in \{\text{'北京'}, \text{'上海'}\} \wedge 2024-01-01 \leq \text{下单日期} \leq 2024-12-31 (K \bowtie H \bowtie D \bowtie P))$ (需加 DISTINCT 去重；连接顺序： $K \bowtie H \rightarrow \bowtie D \rightarrow \bowtie P$)

解析：这题考的是多表 $\bowtie$ 的连接顺序与去重——“从最小关系出发逐步扩到最大关系”。

(1) 查询北京/上海顾客 2024 年下过订单的商品 (去重)

四张表的连接键梳理：

- $K \bowtie H$ ：按 顾客号 连接 ($K. \text{顾客号} = H. \text{顾客号}$)
- $(K \bowtie H) \bowtie D$ ：按 订单号 连接 ($H. \text{订单号} = D. \text{订单号}$)
- $((K \bowtie H) \bowtie D) \bowtie P$ ：按 商品号 连接 ($D. \text{商品号} = P. \text{商品号}$)

完整关系代数表达式：

```

π 商品名,单价 (
σ 城市∈{'北京','上海'} ∧ 2024-01-01 ≤ 下单日期 ≤ 2024-12-31 (
    K ⋈ H ⋈ D ⋈ P
)
)

```

连接顺序说明：从 K (顾客) 出发，先 ⋈ H 拿到"顾客-订单"对，再 ⋈ D 拿到"订单-商品"明细，最后 ⋈ P 拿到商品属性。核心原则——从"维度最小"的关系 (顾客) 出发，逐步 ⋈ 维度更细的关系 (订单 → 明细 → 商品)，避免一开始就做笛卡尔积。

去重处理：

- 关系代数本身有"集合语义" (默认去重)，所以理论上 π 商品名,单价 (...) 已经去重
- 但在 SQL 实现中，由于连接可能产生多份相同结果，必须显式加 DISTINCT

等价的 SQL 写法：

```

SELECT DISTINCT P.商品名, P.单价
FROM K JOIN H ON K.顾客号 = H.顾客号
      JOIN D ON H.订单号 = D.订单号
      JOIN P ON D.商品号 = P.商品号
WHERE K.城市 IN ('北京', '上海')
      AND H.下单日期 BETWEEN '2024-01-01' AND '2024-12-31';

```

易错点：

- 漏写连接条件 → 错。四张表必须两两 ⋈，连接键不能少。
- 城市条件写成 K.城市 = '北京' → 错。是"北京或上海"，必须用 IN 或 城市∈{'北京','上海'}。
- 下单日期写成 下单日期 = '2024' → 严格应写范围 '2024-01-01' 至 '2024-12-31'。
- 漏 DISTINCT → 同名商品被多人下单时会重复出现。

金句：⋈ 的顺序——从"最小关系"出发，逐步扩到"最大关系"；连接键梳理清楚 (外码 = 引用主码)，少一个 ⋈ 条件就变笛卡尔积。

第 3 题

答案：

- (1) π 客户号,姓名,账户总数,账户总余额 (C ⋈ (G 客户号 F COUNT(账户号) AS 账户总数, SUM(余额) AS 账户总余额 (A)))

解析：这题考的是 G 分组 + F 聚合的"先分堆后聚合"流程。

(1) 每位客户的账户总数和账户总余额

G 分组的标准写法 G 分组键 F 聚合函数 AS 新属性名 (关系)：

```

G 客户号 F COUNT(账户号) AS 账户总数, SUM(余额) AS 账户总余额 ( A )

```

关键字表达式分工：

- A 是账户表，输入到 G
- G 客户号：按客户号分堆 (分组键 = 客户号，写在 G 之后)
- F COUNT(账户号) AS 账户总数：每堆内对账户号计数 → 账户总数
- F SUM(余额) AS 账户总余额：每堆内对余额求和 → 账户总余额
- G 之前的"被聚合属性"不需要再单独列出——聚合函数 (F) 已表明这些是被聚合的列

G 与 C 的连接：

- 聚合后的结果只含 客户号, 账户总数, 账户总余额



- 需 $\bowtie$ C 拿到姓名： $(C \bowtie (...))$
- 最后 π 出 4 个属性： π 客户号, 姓名, 账户总数, 账户总余额 (...)

易错点：

- 把 COUNT 写成 COUNT(*) $\rightarrow$ 严格应写 COUNT(账户号) (按账户计数)。
- 把 SUM 写成 SUM(*) $\rightarrow$ 错。SUM 必须对数值列。
- G 后再接 σ 普通选择 $\rightarrow$ 错。G 后只能"按聚合结果选择", 不能在 G 内部 σ 单个账户余额。
- 漏写 COUNT 和 SUM 的"新属性名" $\rightarrow$ 聚合结果没有列名, π 无法引用。

等价的 SQL 写法：

```
SELECT C.客户号, C.姓名, COUNT(A.账户号) AS 账户总数, SUM(A.余额) AS 账户总余额
FROM C JOIN A ON C.客户号 = A.客户号
GROUP BY C.客户号, C.姓名;
```

金句：G 分组的本质——先按 KEY 分堆，再对每堆做 F 聚合；聚合函数必须对"非分组属性"调用，对分组属性调用无意义（每堆内分组属性是常量）。

第 4 题

答案：

- (1) π 学号, 姓名 ($S \bowtie (\pi$ 学号, 课程号 (SC) $\div \pi$ 课程号 (σ 课程名 $\in$ {'数据结构', '离散数学', '操作系统'} (C)))
- (2) π 姓名 ($S \bowtie (\pi$ 学号, 课程号 (SC) $\div \pi$ 课程号 (C)))

解析：这题考的是 $\div$ 除法的"包含所有"语义——"除关系 $\div$ 被除关系 = 包含除关系所有值的元组"。

(1) 选修了"数据库"所有先修课的学生的学号与姓名

题面解读（宽口径，按本题设定）：

- 本题采用宽口径解读——只要求该学生选修了"数据结构"、"离散数学"、"操作系统"这三门先修课
- 不要求该学生也选修"数据库"课本身（即"修读数据库完整链路"为非必要条件）
- 解析末尾附"窄口径"补充写法，供学有余力的同学对照

$\div$ 除法的标准三步构造法：

Step 1：构造被除数——"每个学生选修的(学号, 课程号)对"：

- π 学号, 课程号 (SC)

Step 2：构造除数——"数据库课的所有先修课号"：

- π 课程号 (σ 课程名 $\in$ {'数据结构', '离散数学', '操作系统'} (C))

Step 3：做除法——得到"选修了所有先修课的学生学号"：

- π 学号, 课程号 (SC) $\div \pi$ 课程号 (σ 课程名 $\in$ {'数据结构', '离散数学', '操作系统'} (C))

Step 4：连接 S 拿学号与姓名：

- π 学号, 姓名 ($S \bowtie (\pi$ 学号, 课程号 (SC) $\div \pi$ 课程号 (σ 课程名 $\in$ {'数据结构', '离散数学', '操作系统'} (C)))

被除关系与除关系：

- 被除关系 = π 学号, 课程号 (SC)：每个学生的选课对
- 除关系 = π 课程号 (σ 课程名 $\in$ {...} (C))：先修课号集合

$\div$ 的语义铁律：被除关系 $\div$ 除关系 = 在 SC 中，对学生 x ， x 的"选修课程号"集合 $\supseteq$ 除关系中所有课程号的 x 集合（学号）。即"学生选修了除关系中所有课程"。

窄口径补充写法（若题目改为"必须也选数据库课"）：



```

π 学号,姓名 (
  S ⋈
  (π 学号,课程号 (SC) ÷ π 课程号 (σ 课程名∈{'数据结构','离散数学','操作系统'} (C)))
  ⋈
  σ 课程名='数据库' (C)
)

```

即在宽口径答案末尾 $\bowtie \sigma \text{ 课程名='数据库' (C)}$ 验证该生也选了'数据库'课本身。两种解读的差异在于"是否要求学生也选'数据库'课"——本题采用宽口径，避免窄口径把"只修了先修课但还没修数据库"的同学挡在门外。

(2) 选修了所有课程的学生的姓名

(1) 的特例——除关系 = 所有课程：

Step 1：被除数 = $\pi \text{ 学号,课程号 (SC)}$ (同 (1))

Step 2：除数 = $\pi \text{ 课程号 (C)}$ (所有课程号，无需选择)

Step 3：做除法 $\rightarrow \pi \text{ 学号,课程号 (SC)} \div \pi \text{ 课程号 (C)}$

Step 4：连接 S 拿姓名 $\rightarrow \pi \text{ 姓名 (S} \bowtie (\pi \text{ 学号,课程号 (SC)} \div \pi \text{ 课程号 (C)))}$

易错点：

- 漏写"先选先修课" $\rightarrow$ 错。若直接 $\pi \text{ 课程号 (C)}$ 表示"所有课程"而非"先修课"。(2) 题就是这种用法。
 - $\div$ 写反 $\rightarrow$ 错。 $A \div B$ 是"A 中包含 B 所有值的元组"，不是"B 中包含 A 所有值的元组"。
 - 漏 $\bowtie S$ $\rightarrow$ 错。 $\div$ 结果只有学号，必须再 $\bowtie S$ 才能拿到姓名。
 - 投影列混淆 $\rightarrow$ (1) 要求学号+姓名都要， π 必须列两个属性；(2) 只要姓名， π 只能列姓名。
- 金句： $\div$ 除法的口诀——"包含所有"，被除关系 $\div$ 除关系 = 包含除关系所有值的元组。被除数必须带除数属性（如 学号+课程号），否则类型不匹配。

板块二：关系代数 + SQL 过渡

第 5 题

答案：

- (1) 思路：先找"有教授任教的课程号集合"，再 $\bowtie$ 所有任课信息（不限职称）。

```

π 课程号,课程名,姓名,教室 (
  (π 课程号 (σ 职称='教授' (TC ⋈ T)))
  ⋈ TC ⋈ T ⋈ C
)

```

解析：这题考的是三表 $\bowtie$ 的连接键梳理 + " σ 在 $\bowtie$ 之前"原则的反例——本题不能让 σ 提前吃掉非目标行。

(1) 至少有一位"教授"任教的课程的课程号、课程名、教师姓名、教室

【常见错误】 σ 提前过滤的反例写法：

原速查版写的 $\pi \text{ 课程号,课程名,姓名,教室 (}\sigma \text{ 职称='教授' (C} \bowtie \text{TC} \bowtie \text{T))}$ 是错的——它会把"教授+讲师混合任教的课程中"讲师"那行过滤掉，导致输出不全。

正确思路分两步：

- Step 1：找出"有教授任教的课程号集合"——只关心"哪些课有教授"

- Step 2：用这个课程号集合 $\bowtie$ 所有任课信息（不限职称）——拿到所有任课教师（教授 + 讲师 + 副教授）的清单

完整表达式构造：



Step 1：找有教授任课的课程号（只取课程号，不取教师信息）：

π 课程号 (σ 职称='教授' ($TC \bowtie T$))

- $TC \bowtie T$ 按教师号连接 ($TC.教师号 = T.教师号$)
- σ 职称='教授' 过滤出"教授任课"的 (教师号, 课程号) 对
- π 课程号 只保留课程号

Step 2： $\bowtie$ 所有任课信息（不限职称）：

$(\pi \text{ 课程号 } (\sigma \text{ 职称='教授' } (TC \bowtie T))) \bowtie TC \bowtie T \bowtie C$

- 第一步得到的"有教授的课程号" $\bowtie TC$ ：拿该课所有 (教师号, 课程号) 任课对
- $\bowtie T$ ：加教师信息 (姓名、职称)
- $\bowtie C$ ：加课程信息 (课程名)
- 这样教授+讲师+副教授的任课行都保留

Step 3：投影：

π 课程号, 课程名, 姓名, 教室 (...)

"至少有一位教授"的语义细节：

• 题目要求"至少有一位教授"是筛选条件（必要条件：存在教授），但输出列含"任课教师姓名" → 暗示要保留该课程的所有任课教师

• 若直接 σ 职称='教授' 提前过滤，会丢掉"非教授"任课行 → 输出不全

• 正确做法是"先筛课程号，再 $\bowtie$ 所有任课行"——这是" σ 在 $\bowtie$ 之前"原则的反例：不能让 σ 提前吃掉非目标行

易错点：

• 直接 σ 职称='教授' ($C \bowtie TC \bowtie T$) → 错。会把"教授+讲师混合任课"的课程中"讲师"那行过滤掉，输出不全。

• 用 $\div$ 表达"至少一位" → 错。"至少一位"是 $\exists$ (存在性)，不是 $\forall$ (全称)，用 $\div$ 是杀鸡用牛刀。

• 漏 $\bowtie TC$ → 错。课程和教师之间通过 TC 中间表连接，不能直接 $C \bowtie T$ (外码不匹配，笛卡尔积)

。

• 连接键写错 → 错。 C 与 TC 用课程号， TC 与 T 用教师号，不能混。

• 把 σ 写在最里层 (如 σ 职称='教授' (C)) → 错。 C 表没有"职称"属性， σ 写在 C 上是语法错误。

金句： σ 在 $\bowtie$ 之前——先筛掉无关行，连接成本最低 (通常对)；但当 σ 条件只是"筛选课程"而非"筛选输出行"时，必须把 σ 放在"找课程号"的子查询里，再 $\bowtie$ 所有任课信息——不能让 σ 提前吃掉非目标行。这是" σ 提前优化"的经典反例。

第 6 题

答案：

• (1)

```
SELECT S.姓名, AVG(SC.成绩) AS 平均成绩
FROM S JOIN SC ON S.学号 = SC.学号
WHERE S.所在系 = '计算机系' AND SC.学期 = '2023秋'
GROUP BY S.学号, S.姓名
ORDER BY 平均成绩 DESC;
```

解析：这题考的是 SQL 基础四件套——SELECT / WHERE / GROUP BY / ORDER BY 的执行顺序与"翻译关系代数"思路。

(1) 计算机系学生 2023 秋季学期的平均成绩 (按降序)



SQL 标准四步翻译法：

Step 1：先选表（FROM + JOIN）——确定数据源：

```
FROM S JOIN SC ON S.学号 = SC.学号
```

- S 与 SC 按学号连接（SC.学号 是外码，引用 S.学号）

Step 2：加 WHERE 过滤——只保留计算机系 + 2023 秋：

```
WHERE S.所在系 = '计算机系' AND SC.学期 = '2023秋'
```

- WHERE 是分组前的行筛选，先把无关行过滤掉

Step 3：加 GROUP BY 分组——按学号分组：

```
GROUP BY S.学号, S.姓名
```

- 必须按学号分组（一门课多个学生 → 每个学生一组）
- SQL 标准要求 SELECT 中的非聚集列（S.姓名）都出现在 GROUP BY 中
- MySQL 早期可省（自动取首行），但标准 SQL 不行

Step 4：加 SELECT + ORDER BY——输出姓名与平均成绩，按降序：

```
SELECT S.姓名, AVG(SC.成绩) AS 平均成绩  
ORDER BY 平均成绩 DESC;
```

SQL 各子句的执行顺序（铁律）：

```
FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY
```

易错点：

- 漏 S.姓名 在 GROUP BY → 标准 SQL 报错（MySQL 宽松模式下不报，但结果可能错）。
- WHERE 与 HAVING 混淆 → 错。本题没有 HAVING（不需要过滤分组），过滤都用 WHERE。
- 学期写成 SC.学期 = 2023秋 → 错。学期是字符串，必须用单引号 '2023秋'。
- ORDER BY 用列位置 ORDER BY 2 → 不推荐。ORDER BY 平均成绩 DESC 用别名更清晰。
- AVG 加 DISTINCT → 错。AVG 默认对所有行求平均，DISTINCT 反而丢数据。

金句：SQL 的“翻译关系代数”四步——先选表（FROM+JOIN）→ 加 WHERE → 加 GROUP BY → 加 ORDER BY；执行顺序铁律：FROM→WHERE→GROUP BY→HAVING→SELECT→ORDER BY。

板块三：SQL· 提高

第 7 题

答案：

（1）NOT EXISTS 双层嵌套：

```
SELECT 姓名 FROM S  
WHERE NOT EXISTS (  
    SELECT 1 FROM C  
    WHERE C.所属系 = '计算机系'  
    AND NOT EXISTS (  
        SELECT 1 FROM SC  
        WHERE SC.学号 = S.学号 AND SC.课程号 = C.课程号  
    )  
);
```



(2) IN 子查询 (已修正原速查版的 IN+ALL 陷阱) :

```
-- 改法 A : NOT EXISTS 双重否定 (推荐)
SELECT 姓名 FROM S S1
WHERE 姓名 != '张三'
AND NOT EXISTS (
    SELECT 1 FROM SC SCx
    WHERE SCx.学号 = S1.学号
    AND SCx.成绩 <= (SELECT MAX(SC2.成绩) FROM SC SC2
        JOIN S ON SC2.学号 = S.学号
        WHERE S.姓名 = '张三')
);
```

解析：这题考的是嵌套子查询的两大典型场景——NOT EXISTS 双层表达"所有", IN/EXISTS 表达"存在"。

(1) 选修了计算机系开设的所有课程的学生们的姓名

思路：SQL 没有直接对应 $\div$ 的运算符，但可用双层 NOT EXISTS 模拟"所有"：

- "选修了所有计算机系课程" 等价于 "不存在某门计算机系课程他没选修"
- 外层 NOT EXISTS 遍历课程 ("不存在某门课")
- 内层 NOT EXISTS 检查该学生是否选修该课 ("他没选修")

SQL 逐子句解读：

```
SELECT 姓名 FROM S -- 外层：遍历每个学生
WHERE NOT EXISTS ( -- 否定 1：不存在"某门计算机系课程"
    SELECT 1 FROM C
    WHERE C.所属系 = '计算机系' -- 锁定计算机系课程
    AND NOT EXISTS ( -- 否定 2：该学生没选修该课程
        SELECT 1 FROM SC
        WHERE SC.学号 = S.学号 -- 相关条件：当前学生
        AND SC.课程号 = C.课程号 -- 相关条件：当前课程
    )
);
```

双层否定的逻辑：

- 内层 NOT EXISTS = "该学生没选修 C 中的这门课"
- 外层 NOT EXISTS = "不存在一门计算机系的课是该学生没选修的"
- 合并 = "对所有计算机系的课，该学生都选修了"

为什么必须双层：

- 单层 NOT EXISTS 写不出"所有"——只能表达"不存在某读者没借过的所有书的逆"这种混乱语义
- 双层是关键：外层"否类别"、内层"否记录"

易错点：

- 单层 NOT EXISTS → 语义错误 ("没有选过课的学生"而非"选过所有课的学生")。
- 内层漏 S.学号 或 C.课程号 的相关条件 → 退化为独立子查询，结果变成"全校所有学生中是否有未选修某课的人"。

- 用 IN 实现"所有" → IN 是存在性语义，表达不出"所有"。

金句："所有"→双层 NOT EXISTS，外层否类别 (课)、内层否记录 (选修)；NOT EXISTS 比 NOT IN 在 NULL 时更安全。

(2) 成绩高于"张三"所有课程成绩的学生的姓名

【陷阱提示】速查版答案陷阱说明：

原速查版给的 IN + > ALL 写法存在语义错误：




```
-- 原速查版 ( 语义错误 )
SELECT DISTINCT 姓名 FROM S
WHERE 学号 IN (
  SELECT 学号 FROM SC SC1
  WHERE SC1.成绩 > ALL (
    SELECT 成绩 FROM SC SC2
    WHERE SC2.学号 = (SELECT 学号 FROM S WHERE 姓名 = '张三')
  )
);
```

问题：IN + > ALL 的实际语义是——"该学生至少有一门课成绩 > 张三所有课成绩"，而不是题目要求的"该学生每门课成绩都 > 张三所有课成绩"（即每门课都严格超过张三的最好成绩）。

正确改法 A（NOT EXISTS 双重否定，推荐）：

```
SELECT 姓名 FROM S S1
WHERE 姓名 != '张三'
AND NOT EXISTS (
  SELECT 1 FROM SC SCx
  WHERE SCx.学号 = S1.学号
  AND SCx.成绩 <= (
    SELECT MAX(SC2.成绩) FROM SC SC2
    JOIN S ON SC2.学号 = S.学号
    WHERE S.姓名 = '张三'
  )
);
```

改法 A 解读：

- 内层 SELECT MAX(SC2.成绩) ... 计算张三所有课的最高分
- 外层 NOT EXISTS 检查"不存在 S1 学生的一门课成绩 ≤ 张三最高分"
- 合并语义 = "S1 学生的每门课成绩都 > 张三最高分"

改法 B（MIN > MAX 聚合）：

```
SELECT S.姓名 FROM S
WHERE S.姓名 != '张三'
AND S.学号 IN (
  SELECT SC.学号 FROM SC
  GROUP BY SC.学号
  HAVING MIN(SC.成绩) > (SELECT MAX(SC2.成绩) FROM SC SC2
    JOIN S S3 ON SC2.学号 = S3.学号
    WHERE S3.姓名 = '张三')
);
```

改法 B 解读：

- 用 GROUP BY 学号 + HAVING MIN(成绩) 把"每门课都 > X"转化为"该学生最低分 > X"
- 内层 MAX(SC2.成绩) 计算张三最高分
- 合并 = "S 学生的最低分 > 张三最高分"

两种改法的等价性：

- A 用 NOT EXISTS 表达"不存在反例"
- B 用聚合函数把"每行条件"转为"单值比较"
- 两种思路都正确，结果一致

金句：IN+ALL 的陷阱——IN 是"存在性"语义，只要求子查询中至少有一行匹配，并非"每门课都满足"



"。要表达"所有行都满足"，要么用 NOT EXISTS (双重否定)，要么用聚合函数 MIN/MAX (把每行条件转成单值比较)。

第 8 题

答案：

(1) 相关子查询 (NOT EXISTS 表达"不存在一门课成绩 $\leq$ 该课均分")：

```
SELECT 学号 FROM S
WHERE NOT EXISTS (
  SELECT 1 FROM SC SC1
  WHERE SC1.学号 = S.学号
  AND SC1.成绩 <= (
    SELECT AVG(SC2.成绩) FROM SC SC2
    WHERE SC2.课程号 = SC1.课程号
  )
);
```

(2) NOT EXISTS 相关子查询 ("没有挂过任何课" = "不存在一门课成绩 < 60 ")：

```
SELECT 姓名 FROM S
WHERE NOT EXISTS (
  SELECT 1 FROM SC
  WHERE SC.学号 = S.学号 AND SC.成绩 < 60
);
```

解析：这题考的是相关子查询——子查询中引用外层查询的元组 (关联列在 WHERE 中)，子查询逐行重算。

(1) 每门课程成绩都高于该课程平均成绩的学生的学号

思路：相关子查询——内层的 AVG 是按"SC1.课程号"分组的 (即当前那门课的平均分)，外层的每行学生都需要重新算一次。

SQL 逐子句解读：

```
SELECT 学号 FROM S -- 外层：遍历每个学生
WHERE NOT EXISTS ( -- 否定：不存在"某门课成绩  $\leq$  该课均分"
  SELECT 1 FROM SC SC1
  WHERE SC1.学号 = S.学号 -- 相关条件：关联外层学生
  AND SC1.成绩 <= ( -- 该学生这门课成绩  $\leq$  均分
    SELECT AVG(SC2.成绩) FROM SC SC2
    WHERE SC2.课程号 = SC1.课程号 -- 内层：算这门课的均分
  )
);
```

相关子查询的执行逻辑：

- 外层取一个学生 S (如学号 001)
- 内层 SC1 取出该学生所有选课记录
- 对每条 SC1 记录，重新执行子查询"算 SC1 这门课的平均分"
- 若该学生某门课成绩 $\leq$ 该课均分 $\rightarrow$ 存在反例 $\rightarrow$ NOT EXISTS 排除
- 若该学生所有课成绩都 $>$ 该课均分 $\rightarrow$ 不存在反例 $\rightarrow$ NOT EXISTS 通过

关联列的位置：

- SC1.学号 = S.学号 在内层 WHERE 子句出现 $\rightarrow$ 这是相关子查询的标志



- SC2.课程号 = SC1.课程号 在子查询 WHERE 子句出现 → 跨层关联

易错点：

- 漏写 SC1.学号 = S.学号 → 退化为独立子查询，结果变成"全校学生中是否有成绩 ≤ 某课均分的人"，无意义。
- 把 SC2.课程号 = SC1.课程号 写成 SC2.课程号 = S.学号 → 错。学号和课程号类型不同。
- 用 HAVING 替代 NOT EXISTS → 错。HAVING 在 GROUP BY 之后，无法表达"每门课"的细粒度条件。

金句：相关子查询 = 子查询中引用外层表 (SC1.学号 = S.学号)，子查询逐行重算；关联列在 WHERE 子句出现。

(2) 没有挂过任何课程的学生的姓名

思路：等价转化——"没有挂过任何课" = "不存在一门课使该学生成绩 < 60" = NOT EXISTS 相关子查询。

```
SELECT 姓名 FROM S
WHERE NOT EXISTS (
  SELECT 1 FROM SC
  WHERE SC.学号 = S.学号 -- 相关条件：关联外层学生
  AND SC.成绩 < 60      -- 否定：成绩 < 60 即挂科
);
```

NOT EXISTS 的"双重否定"逻辑：

- 内层：检查该学生是否有"成绩 < 60"的记录
- 外层 NOT EXISTS：否定内层——若存在挂科记录则排除该学生
- 合并：保留"不存在挂科记录"的学生 = "所有课成绩 ≥ 60"的学生

关联列：SC.学号 = S.学号 在内层 WHERE 子句出现。

易错点：

- 写成 WHERE SC.成绩 >= 60 → 错。>= 60 配 NOT EXISTS 表达"不存在成绩 ≥ 60 的课"——语义反了。
- 漏写 SC.学号 = S.学号 相关条件 → 退化为"全校学生中是否有挂科的人"，结果错。
- 写成 WHERE 60 <= (SELECT MIN(SC.成绩) FROM SC WHERE SC.学号 = S.学号) → 也能实现，但需用聚合函数；NOT EXISTS 更直观。

金句：NOT EXISTS 双层嵌套 = 双重否定 = "对所有 X，不存在 Y 反例"；内层引用外层 = 相关子查询的标志。

第 9 题

答案：

(1) 账户总余额超过 10 万元的客户 (按总余额降序)：

```
SELECT C.姓名, SUM(A.余额) AS 总余额
FROM C JOIN A ON C.客户号 = A.客户号
GROUP BY C.客户号, C.姓名
HAVING SUM(A.余额) > 100000
ORDER BY 总余额 DESC;
```

(2) 在 ≥ 3 个不同支行开过户的客户 (按支行数降序)：



```
SELECT C.姓名, C.城市, COUNT(DISTINCT A.开户行号) AS 支行数
FROM C JOIN A ON C.客户号 = A.客户号
GROUP BY C.客户号, C.姓名, C.城市
HAVING COUNT(DISTINCT A.开户行号) >= 3
ORDER BY 支行数 DESC;
```

解析：这题考的是 JOIN + GROUP BY + HAVING 的"三件套"——连接后分组，对组筛选。

(1) 账户总余额超过 10 万元的客户

三件套配合：

- JOIN 连接：C ⋈ A 拿到"客户-账户"对（按客户号连接）
- GROUP BY 分组键：C.客户号, C.姓名（每位客户一组）
- HAVING 过滤：对"组"而非"行"——SUM(A.余额) > 100000 只能出现在 HAVING 中
- ORDER BY 排序：按聚合结果（总余额）降序

关键子句对比：

- WHERE：筛"行"（分组前），不能用聚合函数
- HAVING：筛"组"（分组后），必须用聚合函数

易错点：

- 把 SUM(A.余额) > 100000 写在 WHERE 中 → 错。WHERE 不能用聚合函数。
- GROUP BY 漏 C.姓名 → 标准 SQL 报错。
- 漏 ORDER BY 总余额 DESC → 结果顺序不保证。

金句：HAVING 在 GROUP BY 之后——先按 KEY 分组，再过滤"组"而非"行"；WHERE 筛行，HAVING 筛组，两者不可混。

(2) 在 ≥ 3 个不同支行开过户的客户

关键点：题目要求"不同支行"，必须 COUNT(DISTINCT A.开户行号) 去重！

```
SELECT C.姓名, C.城市, COUNT(DISTINCT A.开户行号) AS 支行数
FROM C JOIN A ON C.客户号 = A.客户号
GROUP BY C.客户号, C.姓名, C.城市
HAVING COUNT(DISTINCT A.开户行号) >= 3
ORDER BY 支行数 DESC;
```

DISTINCT 不可省的场景：

- 同一客户在同支行开了多个账户 → 账户表 A 中有 (客户号, 开户行号) 多条记录
- 若不 DISTINCT → 该 (客户号, 开户行号) 组合被多次计数
- DISTINCT 后 → 每个 (客户号, 开户行号) 组合只计一次 → 得到"不同支行数"

GROUP BY 三个字段的理由：

- C.客户号：分组键
- C.姓名：SELECT 中非聚集列，必须出现在 GROUP BY
- C.城市：同上（题目要求"所在城市"作为输出列）

易错点：

- 漏 COUNT(DISTINCT ...) → 错。同支行多个账户会被重复计数，支行数虚高。
- GROUP BY 漏 C.城市 → 标准 SQL 报错。
- 把 "≥ 3" 写成 "> 3" → 错。题目说"≥ 3 个不同支行"，含 3。
- 把 A.客户号 当分组键 → 错。应按 C.客户号（业务标识），且 GROUP BY 的客户号和 SELECT 的客户号必须一致。

金句：DISTINCT 在 COUNT 内 = 去重计数；"不同 X 数"必须 COUNT(DISTINCT X)，否则同 X 多次出现被多次计。



第 10 题

答案：

(1) 至少订过 3 次票的乘客 (按订票次数降序；同次按姓名升序)：

```
SELECT P.姓名, COUNT(DISTINCT B.航班号) AS 订票次数
FROM P JOIN B ON P.乘客号 = B.乘客号
GROUP BY P.乘客号, P.姓名
HAVING COUNT(DISTINCT B.航班号) >= 3
ORDER BY 订票次数 DESC, P.姓名 ASC;
```

(2) 每位乘客累计购票支出 (未订票的乘客显示 0)：

```
SELECT P.乘客号, P.姓名,
       COALESCE(SUM(B.票数 * F.票价), 0) AS 累计金额
FROM P
LEFT JOIN B ON P.乘客号 = B.乘客号
LEFT JOIN F ON B.航班号 = F.航班号
GROUP BY P.乘客号, P.姓名
ORDER BY P.乘客号;
```

解析：这题综合考 LEFT JOIN 保留所有记录 + COALESCE 兜底 NULL + 多字段排序。

(1) 至少订过 3 次票的乘客

DISTINCT 的作用位置：

• COUNT(DISTINCT B.航班号)：按"航班号"去重统计——同一乘客对同一航班的多次订票只算一次
多字段排序：

```
ORDER BY 订票次数 DESC, P.姓名 ASC;
```

- 第一关键字：订票次数降序 (先按主要指标排)
- 第二关键字：姓名升序 (次数相同时按姓名确定顺序，避免结果不稳定)

易错点：

- COUNT(*) 或 COUNT(B.航班号) 不加 DISTINCT → 错。同一航班多次订票被多次计数。
- ORDER BY 单字段 → 同次数乘客顺序不固定，可能导致结果集顺序不稳定。
- 漏 HAVING COUNT(DISTINCT B.航班号) >= 3 → 错。这是筛选"至少 3 次"的关键。

金句：多字段排序——主指标先排 (降序)，副指标兜底 (升序)；"不同 X 数 ≥ N"必须 COUNT(DISTINCT X)。

(2) 每位乘客累计购票支出 (未订票的乘客显示 0)

思路：LEFT JOIN 保留所有乘客 (包括没订票的)，COALESCE 把 SUM 的 NULL 兜底为 0。

LEFT JOIN 的 ON 子句完整性：

```
FROM P
LEFT JOIN B ON P.乘客号 = B.乘客号 -- 第一步：保留所有乘客
LEFT JOIN F ON B.航班号 = F.航班号 -- 第二步：连航班表拿票价
```

- 第一步 LEFT JOIN：以 P 为驱动表，保留所有乘客；没订票的乘客 → B 表侧全 NULL
- 第二步 LEFT JOIN：把 B 侧连到 F (航班表)；未订票乘客 → F 表侧也全 NULL
- 累计金额 = SUM(B.票数 * F.票价)，未订票乘客的 SUM 输入全是 NULL → SUM 忽略 NULL 返 NULL

LL

COALESCE 兜底逻辑：

```
COALESCE(SUM(B.票数 * F.票价), 0) AS 累计金额
```



- SUM 遇 NULL 返 NULL (不是 0 !)
- COALESCE 把 NULL 替换为 0
- 累计金额计算：B.票数 × F.票价，订票时按此累加；未订票时 $SUM(NULL) = NULL \rightarrow COALESCE \rightarrow 0$

为什么必须 LEFT JOIN (不能用 INNER JOIN) :

- INNER JOIN 会过滤掉没订票的乘客
- 题目要求"未订票的乘客也要列出" → 必须 LEFT JOIN 保留悬浮元组

易错点：

- 用 INNER JOIN → 未订票乘客丢失。
- 漏 COALESCE → 未订票乘客的累计金额显示 NULL 而非 0。
- 聚合时机错 (先 COALESCE 再 SUM) → 错。必须先 SUM (聚合) 再 COALESCE (兜底) 。
- ON 子句漏条件 → 退化为笛卡尔积，所有乘客的累计金额爆炸。

金句：LEFT JOIN + COALESCE = "兜底神器"——没匹配上的显示 0，永不丢记录。"所有 X 都列出"必用外连接 + COALESCE 兜底。

